TH Bingen University of Applied Sciences

Department 2- Technology, Informatics and Economics

Applied Bioinformatics (B.Sc.)

# Protein classification using CNN model with Embedding

Closing report

Date: 28.02.2025

Author: Anna Beketova

Matriculation number: 2176220

# Contents

List of abbreviations ..... 3
Introduction ..... 4
Material ..... 5
Methods ..... 6
Data preprocessing ..... 6
Embedding ..... 6
Labeling and CNN implementation ..... 6
Results ..... 9
Discussion ..... 11
References ..... 12
List of figures ..... 14
List of tables ..... 14
Declaration of Originality ..... 15
Declaration on Intellectual Property ..... 15

# List of abbreviations

| | |
|---|---|
| CNN | Convolutional Neural Network |
| MCC | Matthews correlation coefficient |
| SL | Sequence length |
| BCE | Binary Cross Entropy |
| Adam | Adaptive Moment Estimation |
| LR | Learning rate |
| SGD | Stochastic Gradient Descent |
| WD | Weight decay |
| DL | Dropout layer |

# Introduction

This report deals with a project, which main goal was to determine proteins that have similar structure and physical properties to the corresponding protein family, using Convolutional Neural Network (CNN) model.

CNN-based model was implemented, using Kidera factor embeddings to convert amino acids into numeric values. Kidera factors are a set of 10 physicochemical properties derived from principal component analysis (Kidera et al., 1985). For labeling, protein sequences were mapped to MapMan bins, a hierarchical classification system used in functional genomics (Thimm et al., 2004). Due to the fact that proteins can belong to multiple functional categories, a multi-label classification was applied, where each sequence was represented as a multi-hot encoded vector.

To improve the performance of the model, max pooling techniques and optimization methods were used. Additionally, dropout regularization and weight decay were applied to address class imbalance and overfitting.

Models' performance was evaluated with visualized training and validation losses during training epochs, and calculating model efficiency metrics such as Precision, Recall, F1-Score and Matthews correlation coefficient (MCC).

The results demonstrated that CNNs classifies protein sequences based on their functional bins effectively, achieving high MCC (>0.90). Key insights into the ability of CNNs to detect patterns in protein sequences were gained from the comparison of models with different hyperparameters and optimizers.

# Material

The information about protein families, such as protein names, protein IDs, and description, are assigned to Mapman Bins, that are organized into trees, where leaf nodes are attributed to actual proteins. Some of proteins have more than one Bin. A specific Bincode was assigned to each protein, based on their location at the tree.

The Bins were used for the experiments in form of two datasets: original Mapman Bins (old dataset), and a dataset, where some Bins were removed, and the largest Bins per root Bins were used (updated dataset). Merging old dataset with reference amino acid sequences, that were provided as FASTA file, resulted in 10,085 protein sequences in 40302 protein sequences with updated one.

For the embedding following 10 Kidera values for each amino acid sequence were used:

- Helix/bend preference
- Side-chain size
- Extended structure preference
- Hydrophobicity
- Double-bend preference
- Partial specific volume
- Flat extended preference
- Occurrence in alpha region
- pK-C
- Surrounding hydrophobicity

The values of the Kidera values for each amino acid were extracted with R package Peptides (Comprehensive R Archive Network (CRAN), 2023).

# Methods

The whole list of used libraries and software is given in README file in GitLab repository of the project.

## Data preprocessing

The Mapman file was preprocessed, as Bincodes and protein IDs were extracted, and an amino acid sequence was assigned to each protein. The labeled file contained ','-symbol as a delimiter. To prevent syntax issues from arising, where a comma would be seen as additional columns, these columns were merged.

## Embedding

Protein sequences were encoded using Kidera factors, which represent amino acids as vectors of 10 numerical features. Each protein sequence was transformed into a matrix, where rows correspond to amino acids and columns represent their respective Kidera factors. To handle varying sequence lengths, padding was applied to ensure uniform dimensions. Defined sequence length (SL) indicates, how long each sequence will be. Sequences that are longer are truncated and shorter ones are filled with zeros until the defined SL. Embedding step was performed with *pad_sequences* from TensorFlow Python library. This step created a 3D array of shape *(num_samples, num_features, sequence_length)*, which is then converted into a tensor and used as an input for CNN models (Figure 1), where:

- *num_features = 10* corresponds to the Kidera factors
- *sequence_length* is the fixed sequence size (e.g., 1500)
- *num_samples* is the number of proteins in the dataset

## Labeling and CNN implementation

Bincodes were used as labels for the training, and multi labeled classification with *torch* Python library was performed due to the fact, that some of proteins have more than one Bin.

CNN consists of 4 convolutional layers and an output layer (Table 1). As activation function Sigmoid was applied due to its known efficient work with multi labeled classification tasks (Draelos, 2019). Binary Cross Entropy (BCE) loss function was applied to estimate training and validation losses during training (BCELoss — PyTorch 2.6 Documentation). As optimizer Adaptive Moment Estimation (Adam) was chosen due to its fast performance, low training costs, and adaptive learning rates (LR) (Kingma & Ba, 2014), which could be important in recognizing patterns in protein sequences. Additionally, experiments with Stochastic Gradient Descent (SGD) optimizer were performed to be compared with Adam and address scenarios where Adam could overfit or struggles to generalize (Gupta et al., n.d.). Different CNN models were implemented, where LR and SL were varied. For some models weight decay (WD) 1e-4 was applied to improve model's regularization (D'Angelo et al., 2023b).

*Table 1 Convolutional layers' parameters*

| Layer | Input feature maps | Output feature maps | Kernel size | Stride |
|---|---|---|---|---|
| 1st | 10 | 32 | 7 | 2 |
| 2nd | 32 | 64 | 5 | 2 |
| 3rd | 64 | 128 | 3 | 2 |
| 4th | 128 | 256 | 3 | 2 |

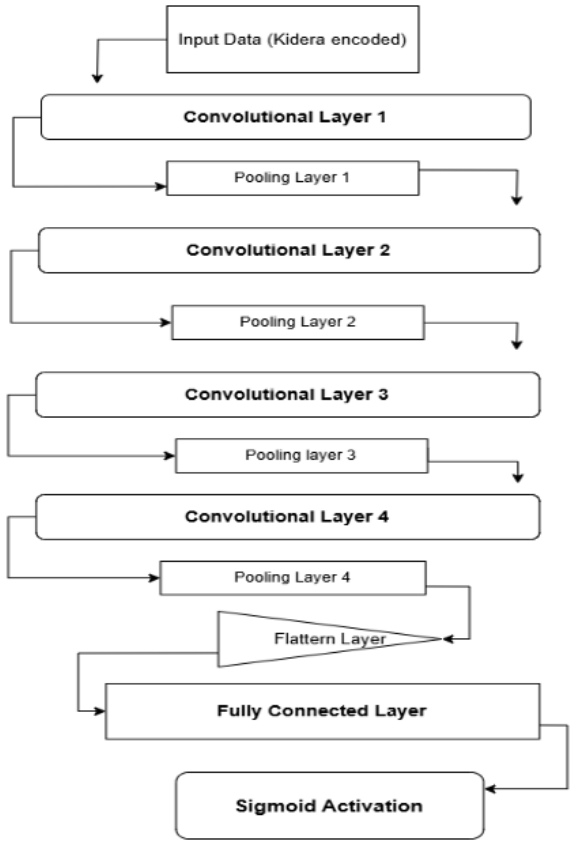


*Figure 1 CNN architecture with pooling layers.*

For the training batch size of 64 was used. The training data was randomly split, so that 80% was used for training and 20% for validation. Max pooling layers after each convolutional layer (kernel size of 2, stride of 2) were applied to improve model's generalization (Zhao & Zhang, 2024). The models were trained for 40 epochs. Additionally, dropout layers (DL) were applied for some experiments to address overfitting further (Dropout layer – Keras Documentation). Information about each experiment is summarized in Table 2.

Cross validation for five folds was performed with SL 1500, Adam optimizer and LR 0.0001 parameters.

*Table 2 Experiments with various SL and hyperparameters of the CNN models*. The experiments were performed with updated dataset. N is the ID number of each experiment. The DL fc column shows DL value before a fully connected layer.

| N | SL | Optimizer | LR | WD | DL1 | DL2 | DL3 | DL4 | DL fc |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1500 | Adam | 0.0001 | None | None | None | None | None | None |
| 2 | 1500 | SGD | 0.1 | 1e-4 | None | None | None | None | None |
| 3 | 1500 | Adam | 0.0001 | 1e-4 | 0.2 | 0.2 | 0.2 | 0.2 | 0.5 |
| 4 | 2500 | Adam | 0.0001 | None | None | None | None | None | None |
| 5 | 2500 | SGD | 0.1 | 1e-4 | None | None | None | None | None |
| 6 | 3500 | Adam | 0.0001 | None | None | None | None | None | None |
| 7 | 3500 | Adam | 0.00001 | None | None | None | None | None | None |
| 8 | 4500 | Adam | 0.0001 | None | None | None | None | None | None |

Experiment using old dataset has the number 0 and was performed with SL 409 and Adam optimizer (LR = 0.001). No max pooling was applied.

Additionally, the training time of experiments, except experiment 0 was measured.

# Results

Labeling using the old dataset resulted in 4937 classes and with the updated one in 30 classes.

Experiment 0 provided low metric values (Table 3). Experiments 1-8, except experiment 7, provided metrics >0.9. Models with Adam optimizer show slightly higher precision and perform slightly faster as SGD ones. Training time increases with SL. Models with longer sequences performed with slightly worse accuracy.

*Table 3 Resulting metrics of the experiments. Training time measured in minutes. For the experiment 0 no MCC and training time were measured.*

| N | Precision | Recall | F1-Score | MCC | Training time |
|---|---|---|---|---|---|
| 1 | 0.9806 | 0.9623 | 0.9712 | 0.9704 | 15.77 |
| 2 | 0.9632 | 0.9082 | 0.9335 | 0.9328 | 17.34 |
| 3 | 0.9825 | 0.9369 | 0.9585 | 0.9581 | 19.63 |
| 4 | 0.9800 | 0.9578 | 0.9687 | 0.9678 | 21.67 |
| 5 | 0.9691 | 0.9026 | 0.9338 | 0.9334 | 29.56 |
| 6 | 0.9725 | 0.9529 | 0.9621 | 0.9610 | 30.70 |
| 7 | 0.9128 | 0.7698 | 0.8300 | 0.8399 | 32.94 |
| 8 | 0.9796 | 0.9638 | 0.9715 | 0.9707 | 41.51 |
| 0 | 0.0247 | 0.1571 | 0.0426 | - | - |

Cross validation showed that metrics remained constant during the folds (Figure 2).

```
Fold 1/5
Fold 1 - Train Loss: 0.0012, Val Loss: 0.0135
Precision: 0.9767, Recall: 0.9606, F1: 0.9684, MCC: 0.9675
Fold 2/5
Fold 2 - Train Loss: 0.0006, Val Loss: 0.0150
Precision: 0.9796, Recall: 0.9625, F1: 0.9707, MCC: 0.9699
Fold 3/5
Fold 3 - Train Loss: 0.0010, Val Loss: 0.0105
Precision: 0.9810, Recall: 0.9614, F1: 0.9708, MCC: 0.9701
Fold 4/5
Fold 4 - Train Loss: 0.0015, Val Loss: 0.0121
Precision: 0.9820, Recall: 0.9603, F1: 0.9709, MCC: 0.9700
Fold 5/5
Fold 5 - Train Loss: 0.0010, Val Loss: 0.0128
Precision: 0.9792, Recall: 0.9581, F1: 0.9684, MCC: 0.9675

Cross-Validation Results:
Average Train Loss: 0.0011
Average Val Loss: 0.0128
Average Precision: 0.9797
Average Recall: 0.9606
Average F1 Score: 0.9698
Average MCC: 0.9690
```

*Figure 2 Metrics of Cross validation experiment.*

Training and validation losses during each epoch show similar tendencies in experiments 1, 3, 6 and 8, as much as in experiments 2, 5 and 7 (Figure 3, a-h). Experiments 4 and 8 have a little bit larger distance between training and validation loss curves after 25$^{th}$ epoch. Experiment 0 shows unstable course of training and validation losses (Figure 3, i).

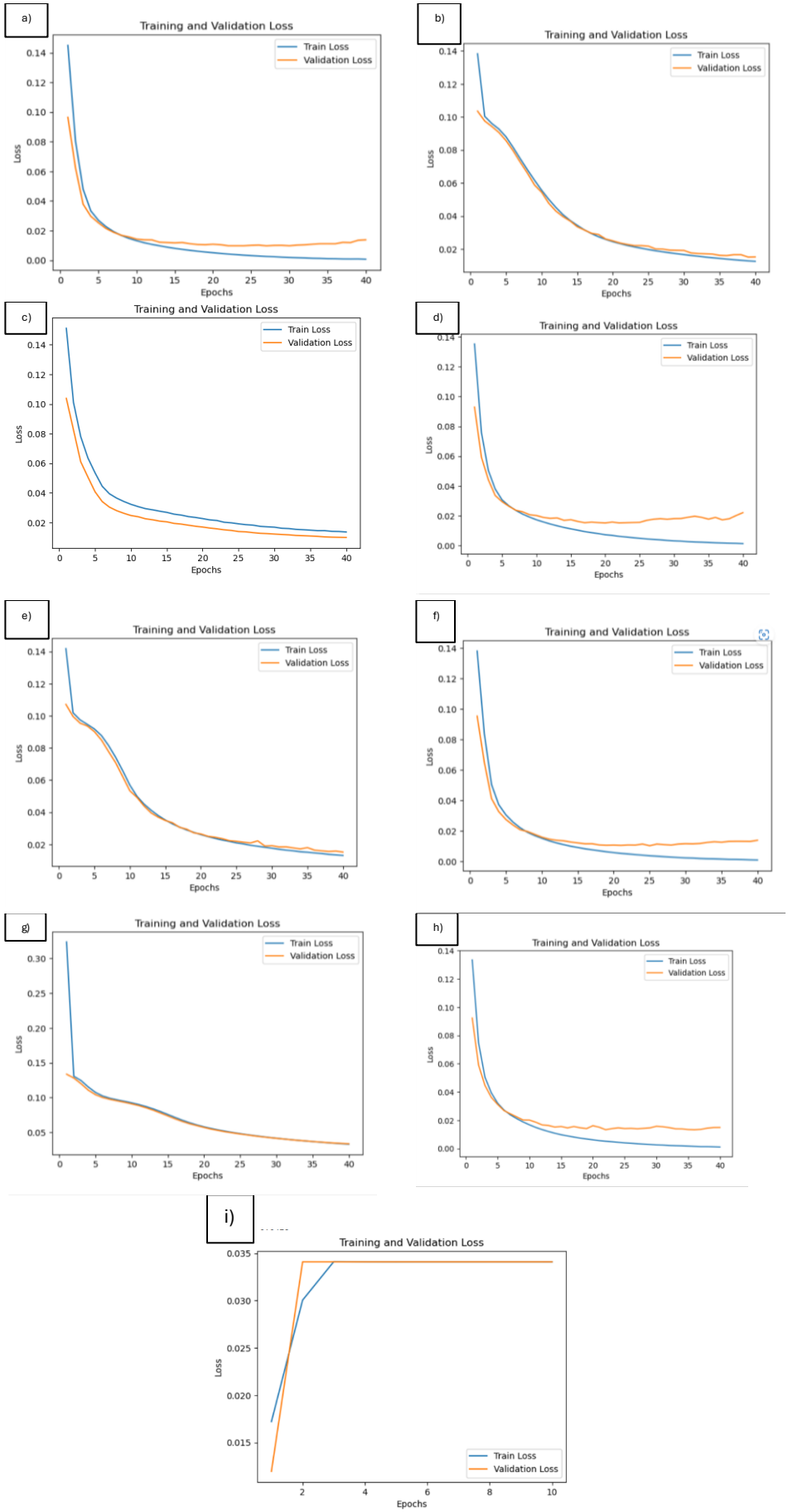


*Figure 3 Training and validation losses graphs for experiments 1-8 (a-h) and for 0 (i).*

# Discussion

The exact reasons of getting low accuracy results in experiment 0 remain unclear. Old dataset has a significant large number of classes, and no Max pooling was applied to the model, which could lead to class imbalance, where some classes have much more samples than others, resulting in model bias toward predicting the majority classes (A Review On Imbalanced Learning Methods). Furthermore, experiment 0 was performed with outstandingly larger LR than other, which could possibly be the reason of unstable results (Brownlee, 2020).

The models with updated reduced dataset, implemented for the experiments 1-8, performed overall efficient training. Adam optimizer models appear to predict proteins slightly better than SGD ones. While SGD generalizes well, avoiding almost completely the overfitting issue, which is shown with training and validation losses moving parallel during almost the whole training period (Figure 3, b,e,g.), Adam models tend to perform training faster and deliver precise results (Table 3), but start overfitting from epochs 15-20 (Figure 3, a,d,f,h). In the end both approaches are suitable for protein classification, since they are both precise and address different situations that occurs during the training. For fast performance with high predictive accuracy Adam can be used, while SGD is better, if generalization and avoiding overfitting are prioritie s.

Combination of DLs and WD with Adam reduces overfitting significantly (Figure3 c). However, the prediction accuracy is slightly worse than of the model without DL and WD (Table 3. Experiments 1 and 3). Dropout ensures better generalization, removing some information during training, but that can slightly influence precision in a negative way (Srivastava et al., 2014). Weight decay forces the model smaller weight values, so its ability to memorize exact training details is limited (Zhang et al., 2018c). Therefore, these parameters are not crucial for the model, if no overfitting issues exist.

Variations in sequence length influence the prediction accuracy of the models. Too long sequences reached with padding may cause noise or redundant information for the training, which complicates the learning process (Rio et al., 2020). Truncating of protein sequences leads to loss of information about specific protein features, but it does not influence the training results negatively. For short sequences it can lead to generalized embeddings and better information gain for the training (Sadeh et al., 2022). This is demonstrated with the metrics of experiments 1 and 8 (Table 3). Although the difference between the results is mild, it is to conclude that experiment 1 delivered the best results judging on the model accuracy, compared with other implemented models during this project.

Positive results from performed experiments with various parameters indicates that CNNs effectively classify proteins using Mapman Bin groups, achieving high accuracy and generalization. Sequence padding improved efficiency without major information loss, and regularization techniques prevented overfitting in some experiments. Kidera factor embeddings captured biochemical properties well. However, CNN approach could be explored further, trying other hyperparameters and larger datasets.

# References


- A Review on Imbalanced Learning Methods. (o. D.). https://www.ijcaonline.org/proceedings/ncac2015/number2/23366-5029/
- BCELoss — PyTorch 2.6 documentation. (o. D.). https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html
- Brownlee J. Understand the impact of learning rate on neural network performance. Machine Learning Mastery. (September 12, 2020). https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/
- Comprehensive R Archive Network (CRAN). (2023, 13. Dezember). Calculate Indices and Theoretical Physicochemical Properties of Protein Sequences [R package Peptides version 2.4.6]. https://cran.r-project.org/web/packages/Peptides/
- D'Angelo, F., Andriushchenko, M., Varre, A., & Flammarion, N. (2023b, October 6). Why do we need weight decay in modern deep learning? arXiv.org. https://arxiv.org/abs/2310.04415v2
- Draelos, V. A. P. B. R., MD PhD. (2019, 8. September). Multi-label vs. Multi-class Classification: Sigmoid vs. Softmax. Glass Box. https://glassboxmedicine.com/2019/05/26/classification-sigmoid-vs-softmax/
- Gupta, A., Ramanath, R., Shi, J., Keerthi, S. S., & Linkedin. (n.d.). Adam vs. SGD: Closing the generalization gap on image classification. In 13th Annual Workshop on Optimization for Machine Learning. Sunnyvale, CA. https://opt-ml.org/papers/2021/paper53.pdf
- Keras documentation: Dropout layer. https://keras.io/api/layers/regularization_layers/dropout/
- Kidera, A., Konishi, Y., Oka, M., Ooi, T., & Scheraga, H. A. (1985). Statistical analysis of the physical properties of the 20 naturally occurring amino acids. Journal of Protein Chemistry, 4(1), 23-55.
- Kingma, D. P., & Ba, J. (2014, December 22). Adam: A method for stochastic optimization. arXiv.org. https://arxiv.org/abs/1412.6980
- Pearce, R., & Zhang, Y. (2021). Deep learning techniques have significantly impacted protein structure prediction and protein design. Current opinion in structural biology, 68, 194–207. https://doi.org/10.1016/j.sbi.2021.01.007
- Rio, A. L., Martin, M., Perera-Lluna, A., & Saidi, R. (2020). Effect of sequence padding on the performance of deep learning models in archaeal protein functional prediction. Scientific Reports, 10(1). https://doi.org/10.1038/s41598-020-71450-8
- Sadeh, G., Wang, Z., Grewal, J., Rangwala, H., & Price, L. (2022, November 9). Training self-supervised peptide sequence models on artificially chopped proteins. arXiv.org. https://arxiv.org/abs/2211.06428
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., Department of Computer Science, & University of Toronto. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting* (Vol. 15, pp. 1929–1958) [Journal-article]. https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf

- Thimm, O., Bläsing, O., Gibon, Y., Nagel, A., Meyer, S., Krüger, P., Selbig, J., Müller, L. A., Rhee, S. Y. & Stitt, M. (2004). mapman: a user-driven tool to display genomics data sets onto diagrams of metabolic pathways and other biological processes. The Plant Journal, 37(6), 914–939. https://doi.org/10.1111/j.1365-313x.2004.02016.x
- Zhang, G., Wang, C., Xu, B., & Grosse, R. (2018b, October 29). Three mechanisms of weight decay regularization. arXiv.org. https://arxiv.org/abs/1810.12281
- Zhao, L., & Zhang, Z. (2024). A improved pooling method for convolutional neural networks. Scientific Reports, 14(1). https://doi.org/10.1038/s41598-024-51258-6

# List of figures

Figure 1 CNN architecture with pooling layers. ..... 7
Figure 2 Metrics of Cross validation experiment. ..... 9
Figure 3 Training and validation losses graphs for experiments 1-8 (a-h) and for 0 (i)... 10

# List of tables

Table 1 Convolutional layers' parameters ..... 7
Table 2 Experiments with various SL and hyperparameters of the CNN models.. ..... 8
Table 3 Resulting metrics of the experiments..... 9

# Declaration of Originality

I declare that the submitted work is the original, which was written by me alone without any help from another person. When the work of others was referenced or used, I made this visible. This work has not been published or graded before.

Bingen am Rhein, 28.02.2025

# Declaration on Intellectual Property

I consent that the Technical University of Applied Sciences Bingen may make this work available to students or third persons as well as publish it under my name (Anna Beketova).

Bingen am Rhein, 28.02.2025